

Exam22 参考答案

姓名: Copilot 学号: 123456

2025 年 12 月 4 日

第 1 题 (8 分)

题目: 求解递归方程的 $T(n)$ 上界: $T(n) = 2T(\frac{n}{2}) + \frac{3}{2}n \log(n-1)$

解答:

我们使用主定理来求解递归方程。首先分析递归方程的形式:

$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{3}{2}n \log(n-1)$$

对于主定理的标准形式 $T(n) = aT(n/b) + f(n)$, 这里:

- $a = 2, b = 2$
- $f(n) = \frac{3}{2}n \log(n-1) \approx \frac{3}{2}n \log n$
- $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

比较 $f(n)$ 与 $n^{\log_b a}$:

$$f(n) = \frac{3}{2}n \log n \quad \text{vs} \quad n^{\log_b a} = n$$

显然 $f(n) = \Theta(n \log n)$, 而 $n^{\log_b a} = n$ 。

由于 $f(n) = \frac{3}{2}n \log n = \Omega(n^{1+\epsilon})$, 其中 ϵ 接近 0 (严格来说这里需要用主定理的第三种情况), 我们需要检查正则条件。

实际上, $f(n) = \frac{3}{2}n \log n$ 比 n 多了一个 $\log n$ 因子, 这对应主定理的第三种情况: 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$ 对某个 $\epsilon > 0$ 成立, 且 $af(n/b) \leq cf(n)$ 对某个 $c < 1$ 和足够大的 n 成立, 则 $T(n) = \Theta(f(n))$ 。

但在本题中, $f(n) = \frac{3}{2}n \log n$ 并不满足 $\Omega(n^{1+\epsilon})$ 的严格形式。因此使用主定理的扩展形式:

当 $f(n) = \Theta(n^{\log_b a} \log^k n)$ 时, 其中 $k \geq 0$, 则:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

在本题中, $k = 1$, 因此:

$$T(n) = \Theta(n \log^2 n)$$

结论: 递归方程的上界为 $O(n \log^2 n)$ 。

第 2 题 (10 分)

题目：概念题

(1) 主定理内容

主定理 (Master Theorem):

设 $a \geq 1$, $b > 1$ 为常数, $f(n)$ 为函数, $T(n)$ 在非负整数上由递归式定义:

$$T(n) = aT(n/b) + f(n)$$

则 $T(n)$ 有如下渐近界:

情况 1: 若 $f(n) = O(n^{\log_b a - \epsilon})$ 对某个常数 $\epsilon > 0$ 成立, 则 $T(n) = \Theta(n^{\log_b a})$

情况 2: 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \log n)$

更一般地, 若 $f(n) = \Theta(n^{\log_b a} \log^k n)$, 其中 $k \geq 0$, 则 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

情况 3: 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$ 对某个常数 $\epsilon > 0$ 成立, 且 $af(n/b) \leq cf(n)$ 对某个常数 $c < 1$ 和所有足够大的 n 成立 (正则条件), 则 $T(n) = \Theta(f(n))$

(2) NPC 问题的概念

NPC (NP-Complete) 问题:

NPC 问题是指同时满足以下两个条件的问题:

1. **该问题属于 NP 类:** 即如果给定一个问题的解 (证书), 可以在多项式时间内验证这个解的正确性。
2. **该问题是 NP-Hard 的:** 即 NP 类中的任何问题都可以在多项式时间内归约到该问题。换句话说, 如果能在多项式时间内解决这个问题, 就能在多项式时间内解决所有 NP 问题。

NPC 问题的重要性:

- NPC 问题是 NP 类中“最难”的问题
- 如果任何一个 NPC 问题能在多项式时间内求解, 则 $P=NP$
- 已知的 NPC 问题包括: SAT 问题、旅行商问题 (TSP)、图着色问题、背包问题等
- 对于 NPC 问题, 目前没有已知的多项式时间算法, 通常采用近似算法、随机算法或启发式算法求解

第 3 题 (11 分)

题目：某个集成电路代工厂收到不同来源的 n 个订单 (a_i, b_i) ，其中 a_i 和 b_i 都是正整数 ($1 \leq i \leq n$)。第 i 个订单 (a_i, b_i) 要求在时间 b_i 之前加工 a_i 个晶圆。代工厂的生产能力为每个时间单位生产 1 个晶圆。工厂希望拒绝最少个数的订单，并适当的安排剩下的订单使得其生产要求得到满足。试设计一个贪心算法求解上述问题。

求解思路

这是一个经典的任务调度问题。关键思想是：

1. 按照截止时间 b_i 对所有订单进行升序排序
2. 按照排序后的顺序依次考虑每个订单
3. 维护当前已完成的工作量和时间
4. 如果当前订单能够在截止时间前完成，则接受；否则拒绝

贪心策略：优先处理截止时间早的订单。这样可以为后续订单留出更多的时间空间。

贪心算法的正确性证明

贪心选择性质：

假设存在最优解，其中某两个被接受的订单 i 和 j 满足 $b_i > b_j$ 但订单 i 在订单 j 之前完成。我们可以交换这两个订单的执行顺序：

设订单 i 在时刻 t_1 开始，订单 j 在时刻 t_2 开始，其中 $t_1 < t_2$ 。

- 原方案：订单 i 在 $[t_1, t_1 + a_i)$ 完成，订单 j 在 $[t_2, t_2 + a_j)$ 完成
- 交换后：订单 j 在 $[t_1, t_1 + a_j)$ 完成，订单 i 在 $[t_1 + a_j, t_1 + a_j + a_i)$ 完成

由于原方案可行，有 $t_1 + a_i \leq b_i$ 和 $t_2 + a_j \leq b_j$ 。

交换后：

- 订单 j ： $t_1 + a_j < t_2 + a_j \leq b_j$ (满足)
- 订单 i ： $t_1 + a_j + a_i = t_2 + a_i \leq b_i$ (因为 $b_i > b_j \geq t_2 + a_j$ ，所以 $t_2 < b_i - a_j \leq b_i - a_i$ ，即 $t_2 + a_i < b_i$)

因此交换不会使原来可行的方案变得不可行，所以按截止时间排序是最优的。

最优子结构：

如果我们已经按贪心策略选择了前 k 个订单，那么剩下的问题就是在剩余时间内选择尽可能多的订单，这是一个规模更小的子问题，且具有相同的结构。

算法伪代码

Algorithm 订单调度贪心算法

Input: 订单集合 $S = \{(a_1, b_1), (a_2, b_2), \dots, (a_n, b_n)\}$ **Output:** 被接受的订单集合

```
1: 按照截止时间  $b_i$  对订单进行升序排序, 得到排序后的订单序列
2:  $accepted \leftarrow \emptyset$  ▷ 被接受的订单集合
3:  $currentTime \leftarrow 0$  ▷ 当前完成时间
4: for  $i = 1$  to  $n$  do
5:   if  $currentTime + a_i \leq b_i$  then ▷ 如果能在截止时间前完成
6:      $accepted \leftarrow accepted \cup \{i\}$ 
7:      $currentTime \leftarrow currentTime + a_i$ 
8: return  $accepted$ 
```

时间复杂度分析

- 排序: $O(n \log n)$
- 遍历所有订单并决策: $O(n)$

总时间复杂度: $O(n \log n)$

第 4 题 (12 分)

题目: 给定 n 个大于 0 的数, 以及一个和 s 。这 n 个数中任意两个不同的数组成的数对 (x, y) , 该数对的和为 $x + y$ 。请给出一个时间复杂度小于 $O(n^2)$ 的分治算法, 计算所有数对中, 满足和 $x + y < s$ 的数对的个数。

求解思路

这个问题可以使用分治思想结合排序来解决:

核心思想:

1. 首先对数组进行排序
2. 使用双指针技术: 左指针从数组开始, 右指针从数组末尾
3. 如果 $A[left] + A[right] < s$, 则 $A[left]$ 可以与 $A[left + 1], \dots, A[right]$ 中的所有元素形成满足条件的数对
4. 否则, 右指针左移

但题目要求使用分治算法，因此我们可以采用归并排序的思想，在归并过程中统计跨越中点的数对数量。

分治策略：

1. **分解：**将数组分成两半
2. **递归求解：**分别统计左半部分内部、右半部分内部满足条件的数对数
3. **合并：**统计跨越中点的数对（一个在左半部分，一个在右半部分）
4. 在合并过程中同时对数组进行排序，为统计跨越数对提供便利

算法伪代码

Algorithm CountPairs(A, s)

Input: 数组 $A[1..n]$, 和阈值 s

Output: 满足 $x + y < s$ 的数对个数

- 1: $A \leftarrow$ 对数组 A 进行排序
 - 2: **return** DivideAndCount($A, 1, n, s$)
-

Algorithm DivideAndCount($A, left, right, s$)

Input: 已排序数组 $A[left..right]$, 和阈值 s

Output: 满足条件的数对个数

- 1: **if** $left \geq right$ **then**
 - 2: **return** 0
 - 3: $mid \leftarrow \lfloor (left + right) / 2 \rfloor$
 - 4: $count \leftarrow$ DivideAndCount($A, left, mid, s$)
 - 5: $count \leftarrow count +$ DivideAndCount($A, mid + 1, right, s$)
 - 6: $count \leftarrow count +$ CountCrossPairs($A, left, mid, right, s$)
 - 7: **return** $count$
-

Algorithm CountCrossPairs(A , left, mid, right, s)

Input: 已排序数组 $A[\text{left}..\text{mid}]$ 和 $A[\text{mid} + 1..\text{right}]$, 和阈值 s
Output: 跨越中点的满足条件数对个数

```

1:  $count \leftarrow 0$ 
2:  $i \leftarrow \text{left}, j \leftarrow \text{right}$ 
3: while  $i \leq \text{mid}$  and  $j > \text{mid}$  do
4:   if  $A[i] + A[j] < s$  then
5:      $count \leftarrow count + (j - \text{mid})$   $\triangleright A[i]$  与  $A[\text{mid} + 1..j]$  都满足
6:      $i \leftarrow i + 1$ 
7:   else
8:      $j \leftarrow j - 1$ 
9: return  $count$ 

```

时间复杂度分析

设 $T(n)$ 为算法的时间复杂度。

递归方程:

$$T(n) = 2T(n/2) + O(n)$$

其中:

- $2T(n/2)$: 递归处理左右两半
- $O(n)$: CountCrossPairs 的时间复杂度 (双指针遍历一次)

使用主定理: $a = 2, b = 2, f(n) = O(n)$

- $n^{\log_b a} = n^{\log_2 2} = n$
- $f(n) = O(n) = \Theta(n^{\log_2 2})$

根据主定理情况 2: $T(n) = \Theta(n \log n)$

加上初始排序的 $O(n \log n)$, 总时间复杂度为: $O(n \log n) < O(n^2)$

第 5 题 (7 分)

题目: 在数据结构 D 上执行操作序列 $op_1, op_2, \dots, op_n$ 。当 $i = 2^k$ 时, op_i 的代价为 i ; 当 $i \neq 2^k$ 时, op_i 的代价为 2。请选择聚集方法和或记账方法二者之一, 分析该操作序列的摊销时间复杂度上界。

使用聚集方法分析

分析：

计算 n 个操作的总代价，然后除以 n 得到摊销代价。

在 n 个操作中：

- 代价为 i 的操作 ($i = 2^k$): 这样的操作有 $\lfloor \log_2 n \rfloor + 1$ 个, 分别是 $i = 1, 2, 4, 8, \dots, 2^{\lfloor \log_2 n \rfloor}$
- 代价为 2 的操作: 共有 $n - (\lfloor \log_2 n \rfloor + 1)$ 个

总代价计算：

$$\begin{aligned}
 \text{总代价} &= \sum_{k=0}^{\lfloor \log_2 n \rfloor} 2^k + 2 \cdot (n - \lfloor \log_2 n \rfloor - 1) \\
 &= (2^{\lfloor \log_2 n \rfloor + 1} - 1) + 2n - 2\lfloor \log_2 n \rfloor - 2 \\
 &\leq 2n + 2n - 2\log_2 n - 3 \\
 &= 4n - 2\log_2 n - 3 \\
 &= O(n)
 \end{aligned}$$

摊销代价：

$$\text{摊销代价} = \frac{\text{总代价}}{n} = \frac{O(n)}{n} = O(1)$$

使用记账方法分析

分析：

我们为每个操作分配摊销代价，使得任何时刻，累计摊销代价不少于累计实际代价。

摊销代价分配：

- 对每个操作 op_i 分配摊销代价为 4

信用额度分析：

对于操作 op_i ：

- 当 $i \neq 2^k$ 时，实际代价为 2，摊销代价为 4，存入信用 2
- 当 $i = 2^k$ 时，实际代价为 $i = 2^k$ ，摊销代价为 4，需要使用信用 $2^k - 4$

在执行操作 op_{2^k} 之前，已经执行了 $2^k - 1$ 个操作，其中：

- $2^{k-1}, 2^{k-2}, \dots, 2, 1$ 这 k 个操作是 2^j 型操作
- 其余 $2^k - 1 - k$ 个操作存入信用

累积信用为: $2 \times (2^k - 1 - k) = 2^{k+1} - 2 - 2k$

这足以支付操作 op_{2^k} 的额外代价 $2^k - 4$ (当 $k \geq 3$ 时)。

因此, 摊销代价为 $O(1)$ 。

结论: 无论使用聚集方法还是记账方法, 该操作序列的摊销时间复杂度上界均为 $O(1)$ 。

第 6 题 (14 分)

题目: 在一个虚拟现实的世界中, 矩形地图被划分成 $m \times n$ 的网格状, 左下角的网格点标记为 $(0, 0)$ 而右上角的网格点标记为 (m, n) , 其中 m, n 皆为正整数。假设某一智能体从 $(0, 0)$ 出发沿网格线行进到达 (m, n) , 但在网格点 (i, j) 处它只能向上行进或向右行进, 向上行进的代价已知为 a_{ij} , 且 $a_{mj} = +\infty$; 向右行进的代价已知为 b_{ij} , 且 $b_{in} = +\infty$ 。请设计一个动态规划算法在这个网格中为该智能体寻找一条代价最小的行进路线。

求解思路

这是一个典型的网格最短路径问题。关键观察:

1. 从 $(0, 0)$ 到 (m, n) 的任何路径只能向右或向上移动
2. 到达某点 (i, j) 的最小代价路径必定经过 $(i-1, j)$ 或 $(i, j-1)$
3. 可以使用动态规划自底向上构建最优解

状态定义: 设 $dp[i][j]$ 表示从 $(0, 0)$ 到达 (i, j) 的最小代价。

边界条件:

- $dp[0][0] = 0$ (起点代价为 0)
- $dp[i][0] = dp[i-1][0] + b_{i-1,0}$ (只能向右)
- $dp[0][j] = dp[0][j-1] + a_{0,j-1}$ (只能向上)

递归方程

对于 $i > 0$ 且 $j > 0$:

$$dp[i][j] = \min \begin{cases} dp[i-1][j] + b_{i-1,j} & \text{从左边来 (向右移动)} \\ dp[i][j-1] + a_{i,j-1} & \text{从下边来 (向上移动)} \end{cases}$$

注意题目中的约束:

- $a_{mj} = +\infty$: 在最上边界 (m, j) 不能再向上
- $b_{in} = +\infty$: 在最右边界 (i, n) 不能再向右

这些约束自然地限制了在边界上的移动方向。

算法伪代码

Algorithm 最小代价网格路径

Input: 网格尺寸 $m \times n$, 向上代价数组 $a[0..m][0..n-1]$, 向右代价数组 $b[0..m-1][0..n]$

Output: 从 $(0, 0)$ 到 (m, n) 的最小代价

```

1: 初始化  $dp[0..m][0..n]$  为  $+\infty$ 
2:  $dp[0][0] \leftarrow 0$ 
3: for  $i = 1$  to  $m$  do                                     ▷ 初始化第一列
4:    $dp[i][0] \leftarrow dp[i-1][0] + b[i-1][0]$ 
5: for  $j = 1$  to  $n$  do                                     ▷ 初始化第一行
6:    $dp[0][j] \leftarrow dp[0][j-1] + a[0][j-1]$ 
7: for  $i = 1$  to  $m$  do
8:   for  $j = 1$  to  $n$  do
9:      $fromLeft \leftarrow dp[i-1][j] + b[i-1][j]$ 
10:     $fromDown \leftarrow dp[i][j-1] + a[i][j-1]$ 
11:     $dp[i][j] \leftarrow \min(fromLeft, fromDown)$ 
12: return  $dp[m][n]$ 

```

时间复杂度分析

- 初始化第一列: $O(m)$
- 初始化第一行: $O(n)$
- 填充 DP 表: 需要计算 $m \times n$ 个状态, 每个状态需要 $O(1)$ 时间

总时间复杂度: $O(mn)$

空间复杂度: $O(mn)$ (可以优化到 $O(\min(m, n))$ 使用滚动数组)

第 7 题 (9 分)

题目: 监控方案优化: 假设给定一个大型的有线互联网络, 该网络包含 n 个路由器节点, 各个路由器之间采用网络电缆连接, 构成无向图状结构。假设每个路由器节点能

访问到与其连接的所有电缆中的流量。现在为了监控所有链路（电缆）中的网络流量，你编写了一个监控程序，该监控程序需要加载到路由器中，从而分析流经该路由器的信息内容。为了降低对整体网络的影响，你试图只给数目尽量少的路由器加载监控程序就实现监控所有链路流量的目标。请设计一个近似比为 2 的算法实现路由器的选择，给出伪代码，并分析该算法的近似比。

问题分析

这是一个**最小顶点覆盖问题** (Minimum Vertex Cover):

- 网络 = 无向图 $G = (V, E)$
- 路由器 = 顶点，电缆 = 边
- 目标：选择最少的顶点，使得每条边至少有一个端点被选中

最小顶点覆盖问题是 NP-Hard 问题，我们使用贪心近似算法。

算法设计思路

使用**边贪心策略**：

1. 选择一条未被覆盖的边
2. 将该边的两个端点都加入顶点覆盖集合
3. 移除所有与这两个端点相关的边
4. 重复直到所有边都被覆盖

算法伪代码

Algorithm 顶点覆盖近似算法

Input: 无向图 $G = (V, E)$

Output: 顶点覆盖集合 C

```
1:  $C \leftarrow \emptyset$  ▷ 顶点覆盖集合
2:  $E' \leftarrow E$  ▷ 未覆盖边的集合
3: while  $E' \neq \emptyset$  do
4:   从  $E'$  中任选一条边  $(u, v)$ 
5:    $C \leftarrow C \cup \{u, v\}$  ▷ 将边的两个端点加入覆盖集
6:   从  $E'$  中删除所有与  $u$  或  $v$  相关的边
7: return  $C$ 
```

近似比分析

定理：该算法是最小顶点覆盖问题的 2-近似算法。

证明：

设 C^* 为最优顶点覆盖， C 为算法输出的顶点覆盖， A 为算法选择的边集。

关键观察：

1. A 中的边两两不相邻（没有公共端点）

因为每次选择一条边后，会删除所有与其端点相关的边，所以不会再选择与已选边共享端点的边。

2. A 中每条边至少需要被一个顶点覆盖

由于 A 中的边两两不相邻，任意两条边不共享端点，因此最优解 C^* 中必须为 A 中的每条边至少选择一个端点。

因此： $|C^*| \geq |A|$

3. 算法选择的顶点数

算法为每条选中的边选择了两个端点，因此： $|C| = 2|A|$

近似比计算：

$$\frac{|C|}{|C^*|} = \frac{2|A|}{|C^*|} \leq \frac{2|A|}{|A|} = 2$$

因此，该算法的近似比为 2。

时间复杂度： $O(|E|)$ ，因为每条边最多被检查常数次。

第 8 题 (8 分)

题目：假设数组 A 是一个包含 n 个 bit 的位数组。假设 A 里面包含一半 0 一半 1，且 0/1 呈随机排列。因此，任何一个确定性的算法想在 A 里面搜寻到一个 1 都需要 $O(n)$ 的时间。请给出一个拉斯维加斯算法，使得可以以非常高的概率在 $O(\log n)$ 的时间内在 A 中搜寻到一个 1。

算法设计思路

拉斯维加斯算法的特点是：

- 总是返回正确结果（如果返回）
- 运行时间是随机变量

由于数组中有一半是 1，随机选择一个位置，有 50% 的概率找到 1。重复采样可以指数级提高成功概率。

策略：随机采样 $O(\log n)$ 次，每次随机选择一个位置检查是否为 1。

算法伪代码

Algorithm 随机搜索 1 (拉斯维加斯算法)

Input: 位数组 $A[1..n]$, 其中恰好有 $n/2$ 个 1

Output: 返回值为 1 的位置索引

```
1:  $k \leftarrow c \log_2 n$  ▷ 采样次数,  $c$  为常数 (如  $c = 10$ )
2: for  $i = 1$  to  $k$  do
3:    $index \leftarrow$  在  $[1, n]$  中均匀随机选择一个整数
4:   if  $A[index] = 1$  then
5:     return  $index$ 
6: return "未找到" ▷ 极低概率发生
```

复杂度分析

时间复杂度上界:

- 采样次数: $k = c \log n$
- 每次采样: $O(1)$
- 总时间: $O(\log n)$

成功概率分析

每次随机采样找到 1 的概率为 $p = 1/2$ (因为数组中恰好一半是 1)。

采样 k 次都失败的概率为:

$$P(\text{失败}) = (1 - p)^k = \left(\frac{1}{2}\right)^k = \left(\frac{1}{2}\right)^{c \log_2 n} = \frac{1}{2^{c \log_2 n}} = \frac{1}{n^c}$$

因此, 至少一次成功的概率为:

$$P(\text{成功}) = 1 - P(\text{失败}) = 1 - \frac{1}{n^c}$$

结论:

- 当 $c = 10$ 时, 成功概率为 $1 - \frac{1}{n^{10}}$, 这在实际中是极高的概率
- 时间复杂度为 $O(\log n)$
- 这是一个高效的拉斯维加斯算法

第 9 题 (12 分)

题目：数字集成电路的设计流程中，逻辑综合将 RTL 代码综合为由工艺库中的标准单元所构成的门级网表。在标准单元工艺库中，同逻辑功能的门可能有不同尺寸的单位可供选择。而不同尺寸的逻辑门，会具有不同的延迟、功耗、面积，一般来说大尺寸的逻辑门速度更快，但面积更大。假设你已经有了一个函数 `eval()` 可以快速评价任何一个组合逻辑电路门级网表的性能（`eval()` 函数返回值越高视作性能越优）。请设计一个基于遗传算法的优化算法，通过自动调整各个门的尺寸优化电路整体性能。

问题分析

这是一个组合优化问题：

- **优化目标：**最大化电路性能（`eval()` 函数值）
- **决策变量：**电路中每个门的尺寸选择
- **约束：**每个门只能选择该类型可用的尺寸

适合使用遗传算法，因为：

- 搜索空间大（每个门多种尺寸选择）
- 目标函数可计算但可能非凸
- 遗传算法善于在大搜索空间中找到较优解

遗传算法设计方案

1. 编码方案 (Encoding)

染色体表示：每个个体（染色体）表示一个完整的门尺寸配置方案。

假设电路有 m 个门，第 i 个门有 k_i 种尺寸可选。

基因编码：

- 染色体 = $[g_1, g_2, \dots, g_m]$
- $g_i \in \{1, 2, \dots, k_i\}$ 表示第 i 个门选择的尺寸等级

例如，对于 C17 电路（6 个门）：

$$\text{染色体} = [2, 1, 4, 1, 2, 1]$$

表示 U7 选择尺寸 X2，U8 选择尺寸 X1，U9 选择尺寸 X4，等等。

2. 适应度函数 (Fitness Function)

$$\text{fitness}(\text{individual}) = \text{eval}(\text{生成该配置的门级网表})$$

其中 $\text{eval}()$ 函数评估电路性能 (考虑延迟、功耗、面积等因素)。

3. 选择操作 (Selection)

使用**锦标赛选择 (Tournament Selection)**:

- 从种群中随机选择 t 个个体 (如 $t = 3$)
- 选择其中适应度最高的个体
- 重复此过程以选择多个父代

优点: 简单高效, 保持种群多样性。

4. 交叉操作 (Crossover)

使用**单点交叉或均匀交叉**:

单点交叉:

- 随机选择交叉点 $c \in [1, m - 1]$
- 父代 1: $[g_1^{(1)}, \dots, g_c^{(1)} | g_{c+1}^{(1)}, \dots, g_m^{(1)}]$
- 父代 2: $[g_1^{(2)}, \dots, g_c^{(2)} | g_{c+1}^{(2)}, \dots, g_m^{(2)}]$
- 子代 1: $[g_1^{(1)}, \dots, g_c^{(1)} | g_{c+1}^{(2)}, \dots, g_m^{(2)}]$
- 子代 2: $[g_1^{(2)}, \dots, g_c^{(2)} | g_{c+1}^{(1)}, \dots, g_m^{(1)}]$

5. 变异操作 (Mutation)

随机重置变异:

- 以概率 p_m (如 $p_m = 0.1$) 对每个基因进行变异
- 如果变异, 将该门的尺寸随机改为该门类型允许的其他尺寸

例如: $g_i = 2 \rightarrow g_i = \text{random}(\{1, 2, 4, 8\} \setminus \{2\})$

6. 算法流程

Algorithm 遗传算法优化门尺寸

Input: 电路网表结构, 门类型及可选尺寸, `eval()` 函数

Output: 优化后的门尺寸配置

```

1: 初始化种群: 随机生成  $N$  个个体 (如  $N = 100$ )
2: 计算每个个体的适应度
3: for  $generation = 1$  to  $MAX\_GEN$  do
4:    $newPopulation \leftarrow \emptyset$ 
5:   for  $i = 1$  to  $N/2$  do                                ▷ 生成  $N/2$  对父代
6:      $parent1 \leftarrow$  锦标赛选择 ()
7:      $parent2 \leftarrow$  锦标赛选择 ()
8:     if  $random() < p_c$  then                                ▷ 交叉概率, 如  $p_c = 0.8$ 
9:        $(child1, child2) \leftarrow$  交叉 ( $parent1, parent2$ )
10:    else
11:       $(child1, child2) \leftarrow (parent1, parent2)$ 
12:       $child1 \leftarrow$  变异 ( $child1$ )
13:       $child2 \leftarrow$  变异 ( $child2$ )
14:       $newPopulation \leftarrow newPopulation \cup \{child1, child2\}$ 
15:   计算  $newPopulation$  中每个个体的适应度
16:   从  $population \cup newPopulation$  中选择最优的  $N$  个个体作为新种群 (精英保留策略)
17: return 种群中适应度最高的个体

```

算法特点

针对本问题的设计:

1. **编码直观:** 每个基因直接对应一个门的尺寸选择
2. **适应度函数:** 直接使用给定的 `eval()` 函数
3. **交叉保持可行性:** 交叉后的子代仍然是有效的门配置
4. **变异增加多样性:** 随机改变门尺寸, 探索新的配置
5. **精英保留:** 确保最优解不会丢失

参数设置建议:

- 种群大小: $N = 50 \sim 200$

- 最大代数: $MAX_GEN = 100 \sim 500$
- 交叉概率: $p_c = 0.7 \sim 0.9$
- 变异概率: $p_m = 0.05 \sim 0.2$
- 锦标赛大小: $t = 3 \sim 5$

终止条件:

- 达到最大代数
- 或连续若干代（如 20 代）最优解无改进